Olympiad Knowledge-Base Build Playbook

A full pipeline for a database-first IMO training system

Prompt pack, model and tool choices, UI workflow, handoffs, starter schemas, and progress tracking.

Purpose

This document is meant to function as an operating manual. It assumes you are building a living mathematics system: structured JSON entries, a validated import pipeline, a searchable frontend, and a progress engine that improves over time.

What this PDF gives you	What it does not try to do
The end-to-end build plan, exact prompts, handoff rules, starter data contracts, progress tracking, and a recommended stack for each phase.	It is not a giant static textbook. The whole point is to avoid that architecture and build a system that compounds quality over time.

Prepared for the current project scope discussed in chat. Revised layout edition.

Contents

North star

Core principle: do not ask any model to generate the whole handbook or whole site in one shot. Small validated entries beat giant regenerations.

1. Executive summary

The strongest version of this project is not a single giant PDF or textbook. It is a structured mathematics knowledge system with five stable layers: research, architecture, UI design, implementation, and controlled content expansion.

Layer	What it does	Output
Research	Maps the domain: subjects, topics, subtopics, prerequisites, and cross-links.	taxonomy.json + topic map
Architecture	Defines IDs, schemas, validation rules, folder structure, and progress model.	schema pack + build rules
UI design	Designs page structure and component system for browsing and training.	Stitch drafts + Figma refinement
Implementation	Builds the local-first app, validators, loaders, search, and progress views.	React/Vite app
Content expansion	Adds theorem, problem, technique, and solution entries one object at a time.	Validated JSON entries

The system scales because each entry is typed. A theorem is not just text. A problem is not just a question. Each object carries IDs, prerequisites, related entries, difficulty, tags, and progress state.

Default stack. Recommended default stack for this project: GPT-5.4 for architecture, prompt design, JSON generation, and validation; Stitch for UI ideation; Figma for refinement; Antigravity with Gemini 3.1 Pro (High) for implementation; Claude Sonnet 4.6 Thinking only as optional prose polish.

2. Recommended stack

Use as few tools as possible while preserving quality. The practical recommendation is a three-tool core plus one optional polish model.

Phase	Tool	Model	Why
Architecture and schema design	ChatGPT / Codex web	GPT-5.4	Best fit for strict contracts, reasoning, and prompt design.
UI ideation	Stitch	Built-in Stitch generation	Fast first-pass layout generation for a structured app.
Implementation	Antigravity	Gemini 3.1 Pro (High)	Strong fit for multi-step app building and agentic coding.
Optional prose polish	Claude web	Claude Sonnet 4.6 Thinking	Useful only if explanations are mathematically sound but too dry.

Cost control. Do not start with APIs unless you specifically need automation. This can be built cleanly in web products first and automated later.

- Use GPT-5.4 when the task demands exact structure, schema design, or validation judgment.

- Use Stitch when the task is visual ideation and page and component layout.
- Use Antigravity with Gemini 3.1 Pro (High) when the task is implementation of a real codebase.
- Use Claude Sonnet 4.6 Thinking only for selective rewriting, not as the main system architect.

- Do not send architecture work to Stitch.
- Do not send schema design to Antigravity first.
- Do not ask the polish model to define canonical field contracts.
- Do not use a coding agent before the taxonomy and schemas are settled.

3. System architecture

Your knowledge base should be database-first and local-first in v1. Every JSON file is a typed mathematical object. The frontend loads, validates, indexes, and renders them.

Folder structure

```
olympiad-db/
  data/
    system/
      taxonomy.json
      enums.json
    number_theory/
      modular_arithmetic/
        definitions/
        theorems/
        techniques/
        examples/
        problems/
        solutions/
      diophantine_equations/
      valuations/
    algebra/
    combinatorics/
    geometry/
    meta_techniques/
  schemas/
    theorem.schema.json
    technique.schema.json
    problem.schema.json
    solution.schema.json
    progress_record.schema.json
    mastery_profile.schema.json
  progress/
    user-progress.json
    review-queue.json
  docs/
    schema-spec.md
    ui-spec.md
    design-tokens.json
  src/
    app/
    components/
    pages/
    lib/
    validators/
```

Entry type	What it stores
definition	Formal meaning, notation, minimal examples, linked theorems.

Entry type	What it stores
theorem	Statement, intuition, proof, triggers, anti-patterns, linked problems.
technique	Recognition cues, setup patterns, execution steps, traps, examples.
problem	Statement, source, difficulty, techniques, hints ladder, linked solution.
solution	Full method, signals, reflections, linked theorems and techniques.
progress_record	User state per entry: attempts, reviews, confidence, next review.
mastery_profile	Aggregated topic-level ability signals for dashboard views.

Important design rule

Do not dump an entire topic into one JSON file. One file per entry keeps validation, search, linking, and editing clean.

4. Full pipeline

The pipeline below is the operating sequence. Each phase states the tool, model, output, and handoff.

Phase	Tool / model	Output	Handoff
A. Taxonomy	GPT-5.4	Subject / topic / subtopic system + prerequisite graph	taxonomy.json
B. Schema plan	GPT-5.4	Entry classes, required fields, enums, IDs	schema-spec.md + schema pack
C. UI ideation	Stitch	Page layout and component drafts	Stitch drafts
D. UI refinement	Figma	Tokens, spacing, components, state behavior	ui-spec.md + design notes
E. App implementation	Antigravity + Gemini 3.1 Pro (High)	Frontend, validators, loaders, progress system	Working local-first app
F. Entry generation	GPT-5.4	Theorem / problem / solution / technique JSON objects	Validated entry files
G. Validation	GPT-5.4 + local AJV	Math and schema checks	Approved imports
H. Optional polish	Claude Sonnet 4.6 Thinking	Improved wording only	Revised text fields

Phase A - taxonomy

Where: ChatGPT or Codex web
Model: GPT-5.4
Goal: Create the canonical map of the domain before any code is written.

```
You are designing the canonical taxonomy for an Olympiad Mathematics knowledge-base website.
```

```
Your job is to create a strict, scalable taxonomy for:
- Number Theory
- Algebra
- Combinatorics
- Geometry
- Meta-Techniques

Requirements:
1. Create a hierarchy: subject -> topic -> subtopic
2. Use names that are rigorous, standard, and stable over time.
3. Avoid duplicates and vague categories.
4. Mark each topic as: core, extension, specialized.
5. Include prerequisite relationships between topics.
6. Include recommended display order for learning.
7. Include likely cross-links between topics.
8. Make the output suitable for a JSON-driven website.

Output exactly these sections:
A. subject list
B. topic hierarchy
C. prerequisite graph
D. cross-topic link suggestions
E. naming conventions
F. final normalized JSON example called taxonomy.json

Do not write code for the website yet.
Output in highly structured markdown first, then output one final taxonomy.json block.
```

Handoff

Expected output: a readable taxonomy report plus one final taxonomy.json block. Save that JSON to data/system/taxonomy.json.

Phase B - schema and data architecture

Where: ChatGPT or Codex web
Model: GPT-5.4
Goal: Define strict starter schemas and naming rules.

```
Using the olympiad taxonomy already defined, design strict JSON schemas for a database-first
 Olympiad Mathematics knowledge system.

I need schemas for:
- topic
- definition
- theorem
- technique
- example
- problem
- solution
- progress_record
- review_schedule
- mastery_profile

Requirements:
1. Each schema must support rigorous math content.
2. Each schema must support linking to other entries by id.
3. Each schema must support progress tracking.
4. Each schema must support future export to HTML and LaTeX.
5. Each schema must include required vs optional fields.
6. Each schema must include validation rules, enums, and field descriptions.
7. Use stable ids and normalization-friendly structure.
8. Make the schemas realistic for use with JSON Schema validation.
9. Keep naming consistent with taxonomy.json.

Output format:
A. global conventions
B. id conventions
```

```
C. enums
D. one schema section at a time
E. one final recommended folder structure

Do not generate app code yet.
```

Handoff

Expected output: schema-spec.md and the exact rules you will later implement as JSON Schema files.

Phase C - UI ideation in Stitch

Where: Stitch
Model: Stitch default generation
Goal: Generate the first UI draft for the database-driven app.

```
Design a desktop-first web app UI for an Olympiad Mathematics knowledge-base and training
system.

The product is a structured database-driven site, not a simple note-taking app.

Core pages:
1. Home dashboard
2. Subject overview page
3. Topic page
4. Entry page for theorem / definition / technique / example
5. Problem page
6. Progress dashboard
7. Search results page
8. Training mode page
9. Review queue page
10. Import / validation page for JSON entries

Visual style:
- clean, modern, academic
- premium but not flashy
- mathematically serious
- optimized for dense information without feeling cluttered
- strong hierarchy and readability
- good dark mode support
- subtle color coding by subject

Key UI components:
- collapsible subject/topic tree
- breadcrumb navigation
- difficulty badges
- prerequisite chips
- related-entry panel
- trigger-pattern panel
- common mistakes panel
- solution reveal component
- spaced-repetition review card
- progress heatmap
- mastery bars by topic
- problem filter sidebar
- search with tags and structured facets
- entry status labels: draft, reviewed, trusted, polished

Important UX requirements:
- theorem and problem pages must handle long mathematical text
- support LaTeX / math rendering areas
- easy movement between theory and problems
- show prerequisites and linked follow-up topics
- progress should feel motivating but serious
- design for scalability to thousands of entries

Please generate:
1. full page set
```

```
2. reusable component system
3. navigation structure
4. clean spacing and typography system
5. polished desktop web mockups

Also make the design easy to export to Figma and easy to implement in HTML / CSS / JS or
React later.
```

Handoff

Expected output: page drafts in Stitch. Export those to Figma. Do not start coding from memory if the design is still unstable.

Phase D - Figma refinement

This phase is mostly manual. Convert rough AI UI into a reusable system: typography scale, spacing tokens, component variants, interaction notes, and a clean page hierarchy.

- Normalize typography: page title, section title, body, code, labels, badges.
- Create reusable cards: TopicCard, ProblemCard, TheoremCard, ProgressCard.
- Define chips and badges: difficulty, status, prerequisite, source type.
- Specify layout behavior for long math content and side panels.
- Create a short ui-spec.md so implementation is not dependent on screenshots alone.

Handoff

Expected output: Figma file + docs/ui-spec.md + optional design-tokens.json.

Phase E - implementation in Antigravity

Where: Antigravity
Model: Gemini 3.1 Pro (High)
Goal: Build the local-first application.

```
Build a local-first web application for an Olympiad Mathematics knowledge-base.

Tech preferences:
- TypeScript
- React + Vite
- Tailwind CSS
- local JSON file loading
- JSON Schema validation using AJV
- no backend required for v1
- architecture should allow later migration to Supabase or local IndexedDB if needed

I will provide:
- taxonomy.json
- schema specs
- Figma design direction
- design tokens

Core requirements:
1. Subject / topic navigation tree
2. Topic pages with grouped entries
3. Entry pages for theorem / definition / technique / example
4. Problem pages with hints and solutions
5. Search and filtering
6. Progress dashboard
7. Import pipeline for JSON entries
8. Validation errors page
9. Related-links and prerequisites panel
10. Clean component architecture
```

```
Data requirements:
- all content comes from JSON files
- validate every imported file against schema
- reject malformed entries
- maintain an in-memory index for:
  id lookup
  topic grouping
  subject grouping
  tag filtering
  prerequisite links
  related links

Progress requirements:
- store user progress locally
- track per-problem state:
  unseen, attempted, solved_with_hint, solved_independently, reviewed, mastered
- track per-topic mastery
- track spaced review schedule
- generate dashboard summaries

Implementation requirements:
- propose folder structure first
- then implement step by step
- do not skip validation architecture
- do not invent hidden backend assumptions
- keep components modular
- keep data contracts strict
- generate clear code comments where helpful

Workflow:
1. First output the proposed folder structure and implementation plan.
2. Then implement the project incrementally.
3. After each major stage, summarize what was built and what files were created.
```

Handoff

Expected output: the actual codebase. Feed Antigravity the taxonomy, schema spec, and Figma notes. Make it propose structure first before it writes files.

5. Prompt pack for content generation and QA

These are the operational prompts you will reuse every time you add content. The contract is simple: generate one object, validate it, import it, and let the site auto-index it.

Theorem entry generator - paste into GPT-5.4

```
Generate one VALID JSON object for an Olympiad Mathematics knowledge-base.

Entry type: theorem

Hard constraints:
1. Output JSON only.
2. Follow the schema exactly.
3. Do not include comments.
4. Do not include markdown fences.
5. Be mathematically rigorous.
6. Use stable ids.
7. Keep fields concise but content-rich.
8. Every cross-link must be realistic and standard.
9. Difficulty labels must reflect olympiad usefulness, not pure abstract difficulty.
10. The theorem must be pedagogically useful for training.

Required fields to fill:
- id
- type
- subject
- topic
- subtopic
- title
- status
- difficulty_band
- olympiad_relevance
- prerequisites
- statement_latex
- plain_english_summary
- intuition
- proof_outline
- full_proof
- trigger_patterns
- anti_patterns
- common_mistakes
- related_entries
- linked_examples
- linked_problems
- tags
- sources
- created_from_prompt

Topic: [PASTE TOPIC]
Theorem title: [PASTE TITLE]

Return only one valid JSON object.
```

Handoff

Expected output: one theorem JSON object. Save it to data/{subject}/{topic}/theorems/{id}.json.

Technique entry generator - paste into GPT-5.4

```
Generate one VALID JSON object for an Olympiad Mathematics knowledge-base.

Entry type: technique

Output JSON only.

The technique entry must explain:
- what the technique is
- when to suspect it
```

```
- what forms trigger it
- when it is a trap
- what prerequisites are needed
- what classic olympiad use cases exist
- how it connects to related techniques

Required fields:
- id
- type
- subject
- topic
- subtopic
- title
- status
- difficulty_band
- prerequisites
- one_sentence_definition
- recognition_cues
- setup_patterns
- execution_steps
- anti_patterns
- common_mistakes
- canonical_examples
- linked_theorems
- linked_problems
- related_techniques
- tags
- sources
- created_from_prompt

Topic: [PASTE TOPIC]
Technique title: [PASTE TITLE]

Return only one valid JSON object.
```

Handoff

Expected output: one technique JSON object. Save it to data/{subject}/{topic}/techniques/{id}.json.

Problem entry generator - paste into GPT-5.4

```
Generate one VALID JSON object for an Olympiad Mathematics knowledge-base.

Entry type: problem

Output JSON only.

The problem entry must be rigorous and training-oriented.

Required fields:
- id
- type
- subject
- topic
- subtopic
- title
- status
- source_type
- source_name
- difficulty_score
- difficulty_band
- olympiad_level
- prerequisite_entries
- primary_techniques
- secondary_techniques
- statement_latex
- plain_text_statement
- hints_ladder
- common_wrong_paths
- what_you_should_notice
- tags
- linked_solution_ids
```

```
- related_entries
- created_from_prompt

Difficulty scale:
1 = beginner
2 = early foundation
3 = intermediate
4 = strong intermediate
5 = entry olympiad
6 = national olympiad
7 = strong national olympiad
8 = shortlist style
9 = IMO Q1/Q4
10 = IMO Q3/Q6

Topic: [PASTE TOPIC]
Problem statement: [PASTE PROBLEM]
Known source info: [PASTE IF ANY]

Return only one valid JSON object.
```

Handoff

Expected output: one problem JSON object. Save it to data/{subject}/{topic}/problems/{id}.json.

Solution entry generator - paste into GPT-5.4

```
Generate one VALID JSON object for an Olympiad Mathematics knowledge-base.

Entry type: solution

Output JSON only.

The solution must be olympiad-rigorous, readable, and genuinely helpful.

Required fields:
- id
- type
- problem_id
- title
- status
- solution_method_name
- method_summary
- full_solution_latex
- plain_english_walkthrough
- key_observations
- hidden_signals
- alternative_methods
- why_this_method_works
- likely_sticking_points
- postsolve_reflection
- linked_theorems
- linked_techniques
- tags
- created_from_prompt

Problem JSON:
[PASTE PROBLEM JSON]

Return only one valid JSON object.
```

Handoff

Expected output: one solution JSON object. Save it to data/{subject}/{topic}/solutions/{id}.json.

Content validator - paste into GPT-5.4

```
Act as a mathematical content validator for an Olympiad Mathematics knowledge-base.
```

```
I will paste a JSON entry.

Tasks:
1. Check mathematical correctness.
2. Check whether the entry is pedagogically useful.
3. Check whether the topic and subtopic classification are appropriate.
4. Check whether the difficulty level is sensible.
5. Check whether the prerequisite links are reasonable.
6. Check whether the related-entry links are plausible.
7. Check whether anything important is missing.
8. Check for verbosity, vagueness, or hallucinated claims.

Return exactly:
A. PASS or FAIL
B. Critical issues
C. Important improvements
D. Suggested corrected field values
E. A cleaned final JSON object only if repair is straightforward
```

Handoff

Expected output: a QA verdict. If it passes, run local AJV validation too. Mathematical QA and schema QA are different checks and you want both.

Optional prose polish - paste into Claude Sonnet 4.6 Thinking

```
Rewrite the following theorem explanation for an olympiad student.

Goals:
- preserve mathematical correctness
- improve clarity
- improve elegance
- keep it concise
- do not alter the theorem statement
- do not introduce new mathematical claims
- do not remove important caveats

Return:
1. improved plain-English summary
2. improved intuition
3. improved proof outline
4. improved common_mistakes section
```

Handoff

Use this only after the entry already passes GPT-5.4 validation. Replace only the text fields you intentionally want to polish.

6. Starter data contracts

These are practical starter contracts. Implement them as JSON Schema files in code. The goal here is readability and stable field design.

- Use lowercase kebab-case IDs, for example nt-mod-fermat-little-theorem.
- Keep subject names normalized: number_theory, algebra, combinatorics, geometry, meta_techniques.
- Use status values consistently: draft, reviewed, trusted, polished.
- Keep topic and subtopic names stable. Rename them only if you are willing to migrate existing files.
- Use related_entries and prerequisite fields to create your graph. Do not encode graph structure only in folder names.

Theorem entry contract

```
{
  "id": "nt-mod-fermat-little-theorem",
  "type": "theorem",
  "subject": "number_theory",
  "topic": "modular_arithmetic",
  "subtopic": "fermat_euler",
  "title": "Fermat's Little Theorem",
  "status": "reviewed",
  "difficulty_band": "core",
  "olympiad_relevance": "high",
  "prerequisites": ["nt-div-primes", "nt-mod-definition"],
  "statement_latex": "If p is prime and p \nmid a, then a^{p-1} \equiv 1 \pmod p.",
  "plain_english_summary": "Powers modulo a prime repeat in a controlled way for numbers not
   divisible by the prime.",
  "intuition": ["Multiplication by a nonzero residue permutes the nonzero residues modulo
  p."],
  "proof_outline": ["Consider the set 1,2,...,p-1.", "Multiply every element by a modulo
  p.", "Compare products."],
  "full_proof": "...",
  "trigger_patterns": ["prime modulus", "large exponent modulo p"],
  "anti_patterns": ["composite modulus with no coprimality condition"],
  "common_mistakes": ["Applying the theorem when p divides a."],
  "related_entries": ["nt-mod-euler-theorem", "nt-mod-order"],
  "linked_examples": ["nt-mod-example-014"],
  "linked_problems": ["nt-mod-problem-041"],
  "tags": ["mod", "prime", "exponents"],
  "sources": ["standard olympiad theorem"],
  "created_from_prompt": "theorem-generator-v1"
}
```

Technique entry contract

```
{
  "id": "nt-val-lte",
  "type": "technique",
  "subject": "number_theory",
  "topic": "valuations",
  "subtopic": "p_adic_valuation",
  "title": "Lifting The Exponent",
  "status": "reviewed",
  "difficulty_band": "extension",
  "prerequisites": ["nt-div-basic", "nt-val-vp-definition"],
  "one_sentence_definition": "A valuation tool for expressions of the form a^n - b^n or a^n
  + b^n under specific divisibility conditions.",
  "recognition_cues": ["v_p(a^n-b^n)", "x^n-y^n divisibility", "powers with common prime
  factors"],
  "setup_patterns": ["Check the exact LTE hypotheses first.", "Separate odd and even cases
  when needed."],
  "execution_steps": ["Identify p", "Verify divisibility hypotheses", "Apply the correct LTE
   form"],
  "anti_patterns": ["Using LTE without checking the hypotheses"],
  "common_mistakes": ["Forgetting the p=2 caveats"],
  "canonical_examples": ["nt-val-example-003"],
  "linked_theorems": ["nt-val-vp-basic"],
  "linked_problems": ["nt-val-problem-022"],
```

```
  "related_techniques": ["nt-val-order", "nt-val-factorization"],
  "tags": ["valuation", "divisibility", "powers"],
  "sources": ["standard olympiad technique"],
  "created_from_prompt": "technique-generator-v1"
}
```

Problem entry contract

```
{
  "id": "nt-dio-problem-031",
  "type": "problem",
  "subject": "number_theory",
  "topic": "diophantine_equations",
  "subtopic": "descent",
  "title": "Find all integer solutions to ...",
  "status": "reviewed",
  "source_type": "training_set",
  "source_name": "custom set",
  "difficulty_score": 7,
  "difficulty_band": "strong_national_olympiad",
  "olympiad_level": "national_olympiad",
  "prerequisite_entries": ["nt-div-basic", "nt-mod-basic", "nt-descent-intro"],
  "primary_techniques": ["nt-descent"],
  "secondary_techniques": ["nt-factorization", "nt-mod-parity"],
  "statement_latex": "...",
  "plain_text_statement": "...",
  "hints_ladder": ["Look at parity first.", "Try to factor.", "Assume a minimal positive
  solution."],
  "common_wrong_paths": ["Brute forcing residues forever without using structure."],
  "what_you_should_notice": ["The equation has a natural size-reduction mechanism."],
  "tags": ["descent", "integer solutions", "factorization"],
  "linked_solution_ids": ["nt-dio-problem-031-sol-1"],
  "related_entries": ["nt-descent", "nt-factorization"],
  "created_from_prompt": "problem-generator-v1"
}
```

Solution entry contract

```
{
  "id": "nt-dio-problem-031-sol-1",
  "type": "solution",
  "problem_id": "nt-dio-problem-031",
  "title": "Infinite descent solution",
  "status": "reviewed",
  "solution_method_name": "Infinite descent",
  "method_summary": "Assume a minimal positive solution and derive a smaller one.",
  "full_solution_latex": "...",
  "plain_english_walkthrough": [
    "First isolate the divisibility structure.",
    "Then show any nontrivial solution produces a smaller one."
  ],
  "key_observations": ["Minimality is the organizing idea."],
  "hidden_signals": ["A factorization step suggests size reduction."],
  "alternative_methods": ["modular contradiction, if available"],
  "why_this_method_works": "The equation is stable under a size-reducing transformation.",
  "likely_sticking_points": ["Finding the smaller solution cleanly."],
  "postsolve_reflection": ["The main signal was not parity alone but the descent
  structure."],
  "linked_theorems": ["nt-div-basic"],
  "linked_techniques": ["nt-descent", "nt-factorization"],
  "tags": ["descent", "diophantine"],
  "created_from_prompt": "solution-generator-v1"
}
```

Progress record contract

```
{
  "entry_id": "nt-dio-problem-031",
  "entry_type": "problem",
  "status": "solved_with_hint",
  "attempt_count": 2,
  "success_count": 1,
  "last_reviewed_at": "2026-03-07T16:30:00Z",
  "next_review_at": "2026-03-10T16:30:00Z",
  "confidence_score": 0.55,
  "notes": "Recognized descent only after the second hint."
```

```
}
```

Mastery profile contract

```
{
  "subject": "number_theory",
  "topic": "diophantine_equations",
  "mastery_level": 3,
  "problem_solving_level": 3,
  "theory_familiarity": 4,
  "technique_recognition": 2,
  "last_updated_at": "2026-03-07T16:30:00Z"
}
```

Implementation note

If you want strict machine validation, turn these readable contracts into JSON Schema files after your field names are finalized. Do not churn field names after content starts to accumulate.

7. Progress tracking model

Progress tracking is one of the highest-value parts of the system. It turns the site from a library into a training engine.

Layer	What to record	Why it matters
Problem state	unseen, attempted, solved_with_hint, solved_independently, reviewed, mastered	Lets you distinguish exposure from actual competence.
Review schedule	last review, next review, confidence, notes	Supports spaced repetition.
Technique recognition	whether you can spot the cue, not just execute after being told	Olympiad problems are mostly recognition plus execution.
Topic mastery	aggregated theory, solving, and recognition scores	Useful for dashboards and weakness analysis.

Recommended dashboard blocks

- Subject heatmap: number theory, algebra, combinatorics, geometry, meta-techniques.
- Topic mastery bars within each subject.
- Review queue for entries due today.
- Attempt streak and solved-independently streak.
- Weakness panel based on repeated hint dependence or low recognition scores.

Scope control

Keep progress local in v1. You can add sync later. Do not let backend work delay the first usable version.

8. First milestone and operating rules

Best first milestone

- One subject only: Number Theory.
- Three topics: divisibility, modular arithmetic, linear Diophantine equations.

- Working import validation.
- Working topic pages and problem pages.
- Working local progress dashboard.

Rules that protect quality

- Never add content without both schema validation and math validation.
- Never ask a model to write an entire subject at once.
- Never let free-form notes bypass the entry contracts.
- Never change IDs casually after files already link to them.
- Never treat polished wording as a substitute for mathematical correctness.

Final rule

The system wins by compounding quality. Small, typed, reviewed entries give you a base you can trust and expand.

Recommended next action

Run Phase A and Phase B first. Do not touch Antigravity until taxonomy and schemas are stable.

Build order

Minimal sequence: GPT-5.4 taxonomy → GPT-5.4 schema plan → Stitch → Figma → Antigravity with Gemini 3.1 Pro (High) → GPT-5.4 entry generation → GPT-5.4 validation → optional Claude polish.

End of playbook.